

Lesson #10: Unhandled Exceptions

Part 1) Goal and Vulnerability Summary

This lesson is about unhandled exceptions in the DVSA backend Lambda flow. The main affected component is the order-manager Lambda that invokes other Lambda functions and returns their response to the client. The security impact is information disclosure, because raw stack trace and internal error details can be exposed to the user. The main weakness is that the outer function returns the invoked function response without sanitizing error messages.

Part 2) Why This Works / Root Cause

The root cause is bad error handling. The order-manager function parses the downstream Lambda payload and sends it back directly in the HTTP response. When the inner function throws an exception, fields like `stackTrace`, `errorType`, and `errorMessage` are exposed to the client instead of a generic safe message.

Part 3) Environment and Setup

- DVSA order API endpoint: `/dvsa/order`
- AWS services used: API Gateway, AWS Lambda, AWS Console
- Target backend component: order-manager Lambda function
- Tools used: browser DevTools, terminal, code editor, zip, AWS Console

Part 4) Reproduction Steps

1. Log in to DVSA normally and open browser DevTools. In the Network tab, capture a normal POST request to the `/dvsa/order` endpoint.
2. Inspect the order-manager code and confirm that the invoked Lambda payload is returned directly to the client without filtering.
3. Replay the same API request with an incomplete body: `{"action":"get"}`.
4. Observe that the response exposes internal exception details such as `stackTrace`, `errorType`, and `errorMessage`.
5. Edit `order-manager.js` and add safe error handling so downstream exceptions are not returned directly.
6. Zip the updated Lambda package, upload it to AWS Lambda console, and deploy the new code.
7. Repeat the same incomplete request `{"action":"get"}` and verify that the API now returns a generic safe error message instead of internal exception details.

Part 5) Evidence and Proof

FIGURE 1: Normal order-manager API request captured from the browser.

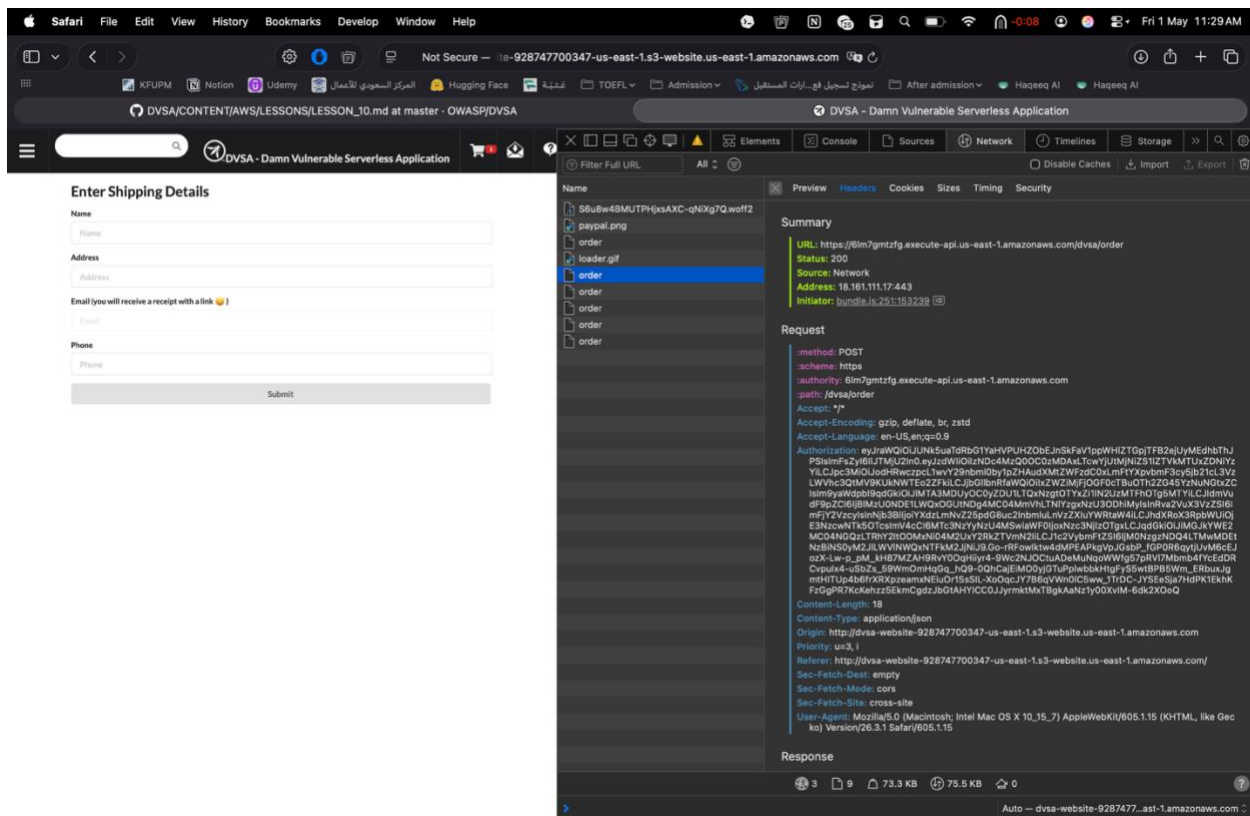


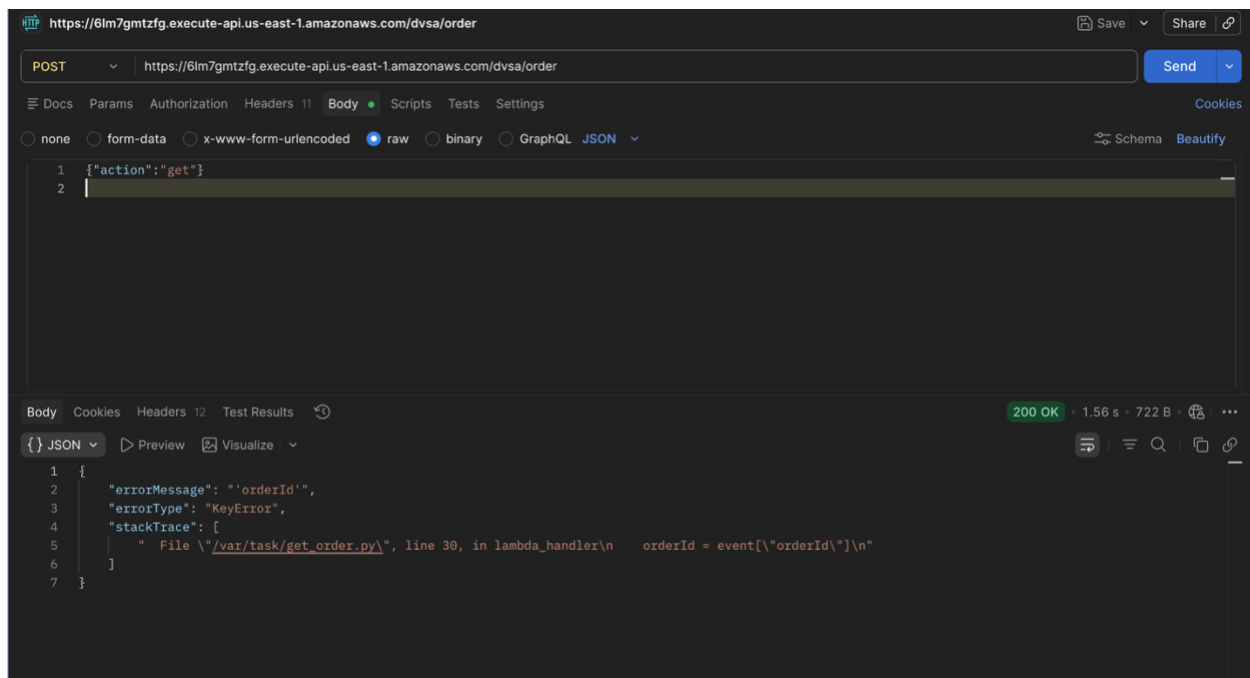
FIGURE 2: order-manager Lambda returning invoked function response directly.

```

169         if (isOk == true) {
170
171             var params = {
172                 FunctionName: functionName,
173                 InvocationType: 'RequestResponse',
174                 Payload: JSON.stringify(payload)
175             };
176             const lambda_client = new LambdaClient();
177             const command = new InvokeCommand(params);
178             lambda_client.send(command).then((lambda_response) => {
179                 const data = JSON.parse(Buffer.from(lambda_response.Payload).toString());
180                 const response = {
181                     statusCode: 200,
182                     headers: {
183                         "Access-Control-Allow-Origin" : "*"
184                     },
185                     body: JSON.stringify(data)
186                 };
187                 callback(null, response);
188             });
189         }

```

FIGURE 3: Raw unhandled exception returned for incomplete get request.



The main proof is the response from the incomplete GET request. It returned internal fields such as `stackTrace`, `errorType`, and `errorMessage`. This proves the backend was exposing implementation details to the client instead of handling the exception safely.

Part 6) Fix Strategy / Probable Mitigation

The fix belongs in the `order-manager` Lambda function. The security improvement is to inspect the invoked Lambda result and block raw exception fields from being returned to the client. Instead of forwarding the full error payload, the function should return a generic safe response and keep internal details out of the external API response.

Part 7) Code / Config Changes

File / resource changed: `backend/functions/order-manager/order-manager.js`

Before, the function parsed `lambda_response.Payload` and returned `JSON.stringify(data)` directly to the client.

After the fix, the function checks for `FunctionError`, `stackTrace`, `errorType`, and `errorMessage`. If any exist, it returns a generic error response instead of the raw backend payload.

The updated logic also adds a catch block for `invoke` failures and returns a safe error message there as well.

FIGURE 4: Safe error handling added to sanitize backend exceptions.

```

169     if (isOk == true) {
170
171         var params = {
172             FunctionName: functionName,
173             InvocationType: 'RequestResponse',
174             Payload: JSON.stringify(payload)
175         };
176         const lambda_client = new LambdaClient();
177         const command = new InvokeCommand(params);
178
179         lambda_client.send(command).then((lambda_response) => {
180             const data = JSON.parse(Buffer.from(lambda_response.Payload).toString());
181
182             if (
183                 lambda_response.FunctionError ||
184                 data.stackTrace ||
185                 data.errorType ||
186                 data.errorMessage
187             ) {
188                 const safeResponse = {
189                     statusCode: 500,
190                     headers: {
191                         "Access-Control-Allow-Origin": "*"
192                     },
193                     body: JSON.stringify({
194                         status: "err",
195                         msg: "request could not be processed"
196                     })
197                 };
198                 return callback(null, safeResponse);
199             }
200
201             const response = {
202                 statusCode: 200,
203                 headers: {
204                     "Access-Control-Allow-Origin": "*"
205                 },
206                 body: JSON.stringify(data)
207             };
208             callback(null, response);
209         }).catch((err) => {
210             const safeResponse = {
211                 statusCode: 500,
212                 headers: {
213                     "Access-Control-Allow-Origin": "*"
214                 },
215                 body: JSON.stringify({
216                     status: "err",
217                     msg: "internal server error"
218                 })
219             };
220             callback(null, safeResponse);
221         });
222     }

```

Part 8) Verification After Fix

After deployment, the same malformed request `{"action":"get"}` no longer returned stack trace or internal file paths. The API returned HTTP 500 with a generic message "request could not be processed", which is safer because it hides backend exception details. The order-manager endpoint still remained reachable after deployment.

FIGURE 5: order-manager Lambda folder before packaging.

```
/Users/rashedjafry/Documents/GitHub/ICS344/DVSA/backend/functions/order-manager
node_modules      order-manager-fixed.zip order-manager.js      package-lock.json   package.json
(base) rashedjafry@MacBookPro order-manager %
```

FIGURE 6: Zipped updated Lambda package for Lesson 10 deployment.

```
(base) rashedjafry@MacBookPro order-manager % zip -r order-manager-lesson10-fix.zip .

adding: node_modules/ (stored 0%)
adding: node_modules/.bin/ (stored 0%)
adding: node_modules/base64url/ (stored 0%)
adding: node_modules/base64url/LICENSE (deflated 40%)
adding: node_modules/base64url/dist/ (stored 0%)
adding: node_modules/base64url/dist/base64url.js (deflated 66%)
adding: node_modules/base64url/dist/base64url.d.ts (deflated 60%)
adding: node_modules/base64url/dist/.gitkeep (stored 0%)
adding: node_modules/base64url/dist/pad-string.js (deflated 54%)
adding: node_modules/base64url/dist/pad-string.d.ts (deflated 14%)
adding: node_modules/base64url/index.js (deflated 29%)
adding: node_modules/base64url/readme.md (deflated 52%)
adding: node_modules/base64url/package.json (deflated 49%)
adding: node_modules/.package-lock.json (deflated 36%)
adding: package-lock.json (deflated 46%)
adding: package.json (deflated 37%)
adding: order-manager-fixed.zip (stored 0%)
adding: order-manager.js (deflated 77%)
(base) rashedjafry@MacBookPro order-manager %
```

FIGURE 7: Target order-manager Lambda function in AWS Console.

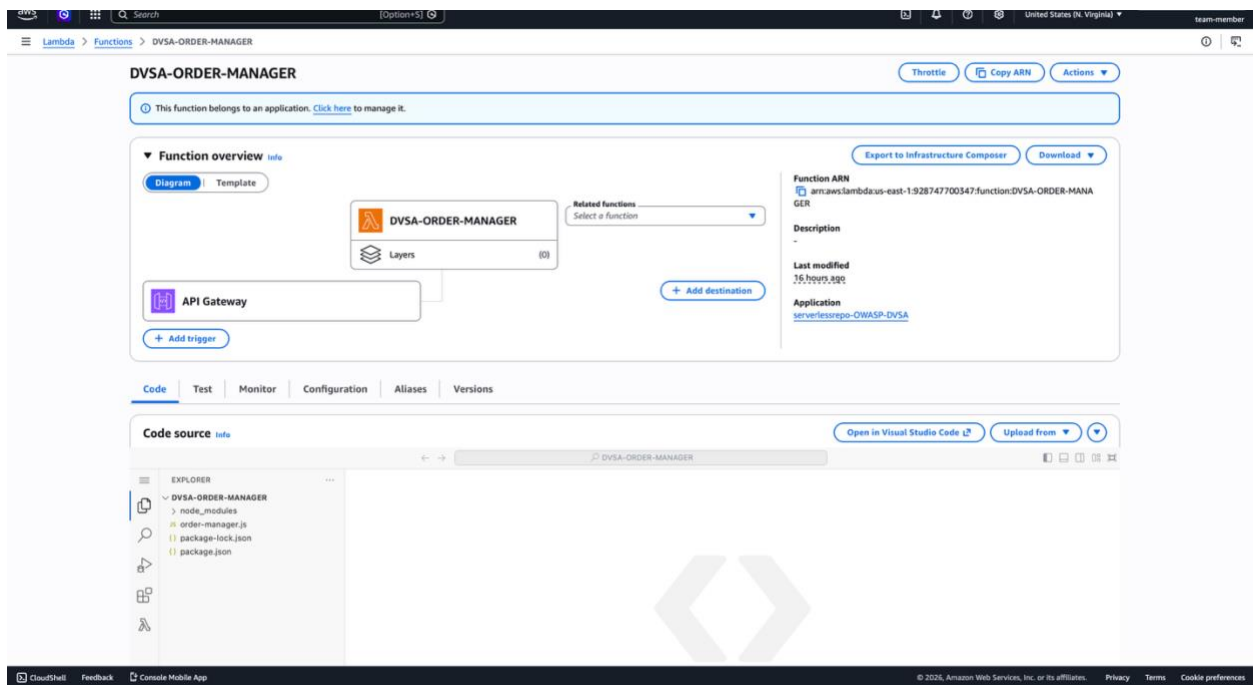


FIGURE 8: Successful deployment of Lesson 10 Lambda fix.

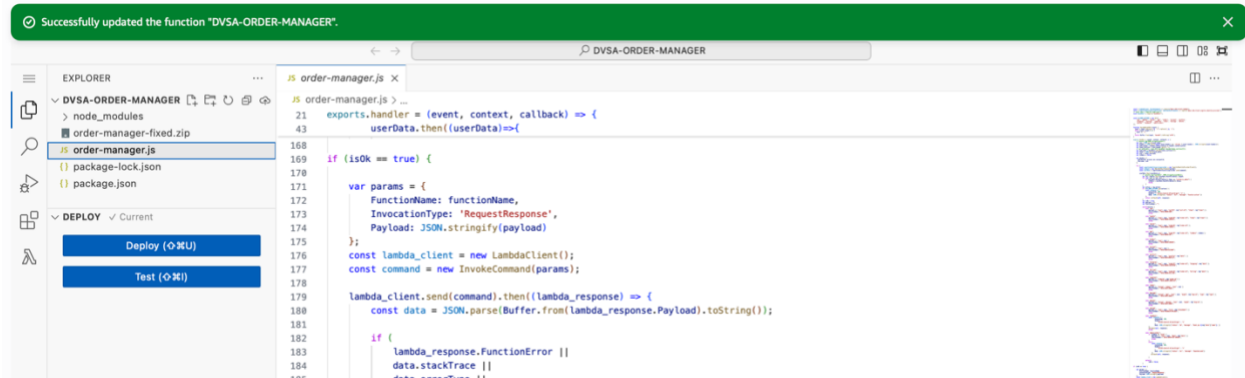
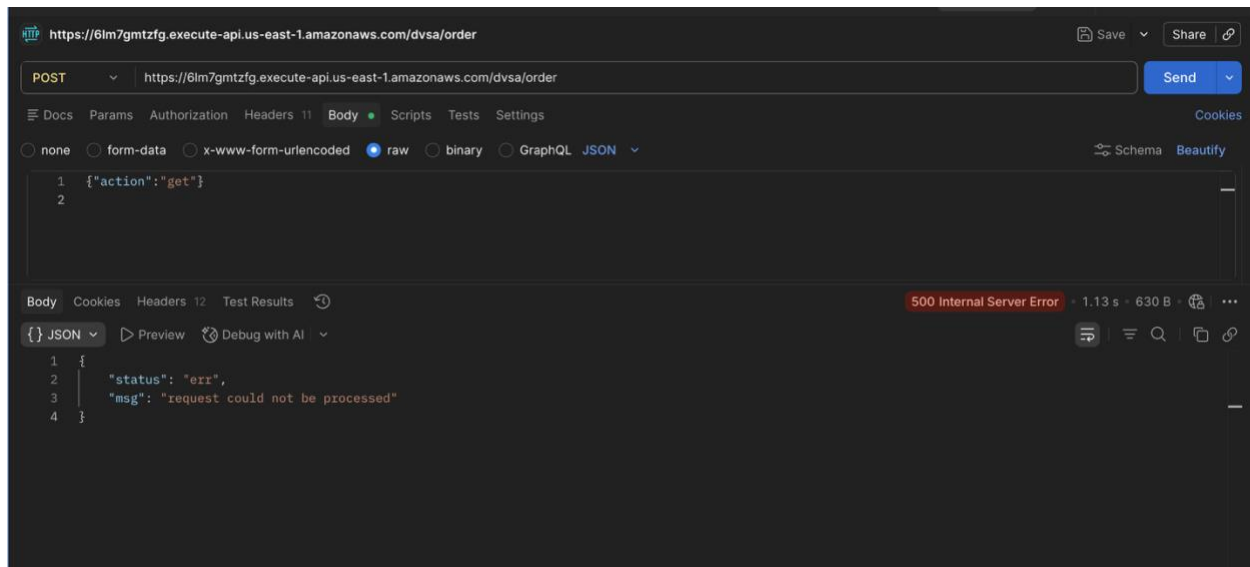


FIGURE 9: Sanitized response after repeating the incomplete get request.



Part 9) Structured Operation and Security Analysis

Table A — Intended logic, artifacts, and behavior comparison

Vulnerability	Intended Rule(s)	Artifacts Used to Infer Rule	Normal Behavior Evidence	Exploit Behavior Evidence
Unhandled Exceptions	The outer Lambda must not expose internal exception details to the client.	order-manager.js, browser Network capture, malformed request response, AWS Console	Normal POST request to /dvsa/order was captured from the browser	Request <code>{\"action\": \"get\"}</code> returned <code>stackTrace</code> , <code>errorType</code> , and <code>errorMessage</code> before the fix

Table B — Deviation analysis, fix, and verification

Vulnerability	Why This Is a Deviation	Deviation Class	Fix Applied (Where)	Post-Fix Verification	Optional Latency Before / After Logging
Unhandled Exceptions	The backend returned raw downstream exception	Misconfiguration	Added sanitized error handling in backend/functions/order-manager/order-manager.js	Same malformed request returned only a generic HTTP 500	Not measured

	data directly to the user.			message after deployment	
--	-------------------------------	--	--	--------------------------------	--

9.1 Intended Logic and Security Rule(s)

The intended logic is that the client sends a request to API Gateway, the order-manager Lambda invokes the correct backend Lambda, and the result is returned to the user. If a backend error happens, the client should receive only a safe generic message. The system must not return internal stack traces, file paths, or backend exception details to the client.

9.2 Evidence Sources and Behavior Trace

The main evidence came from the browser Network capture, the order-manager.js source code, the malformed request response, and AWS deployment proof. In the normal case, the order endpoint accepted browser requests. In the vulnerable case, the incomplete get request caused a raw internal exception response. After the fix, the same request returned a generic safe message instead.

9.3 Deviation Analysis and Classification

This behavior is a deviation because internal backend exception data should stay inside logs and not be exposed to users. The raw response proved that the system leaked implementation details such as `stackTrace` and `errorMessage`. I classify this as misconfiguration because the main problem was unsafe response handling in the outer Lambda.

9.4 Explainable Fix and Post-Fix Validation

The wrong assumption was that the invoked Lambda response can be sent back directly in all cases. The fix was added in `order-manager.js` to detect error responses and replace them with a generic safe message. Post-fix validation showed that the same request `{"action":"get"}` returned HTTP 500 with "request could not be processed" and no internal stack trace.

Part 10) Takeaway / Lessons Learned

The main problem was trusting raw backend exception output and returning it to the client. This is important in serverless systems because one Lambda can leak details from other internal functions and services. A good design rule is to sanitize all backend errors, keep detailed logs internally, and return only safe messages to users.